INFORME TÉCNICO · MOOPING/BACKEND

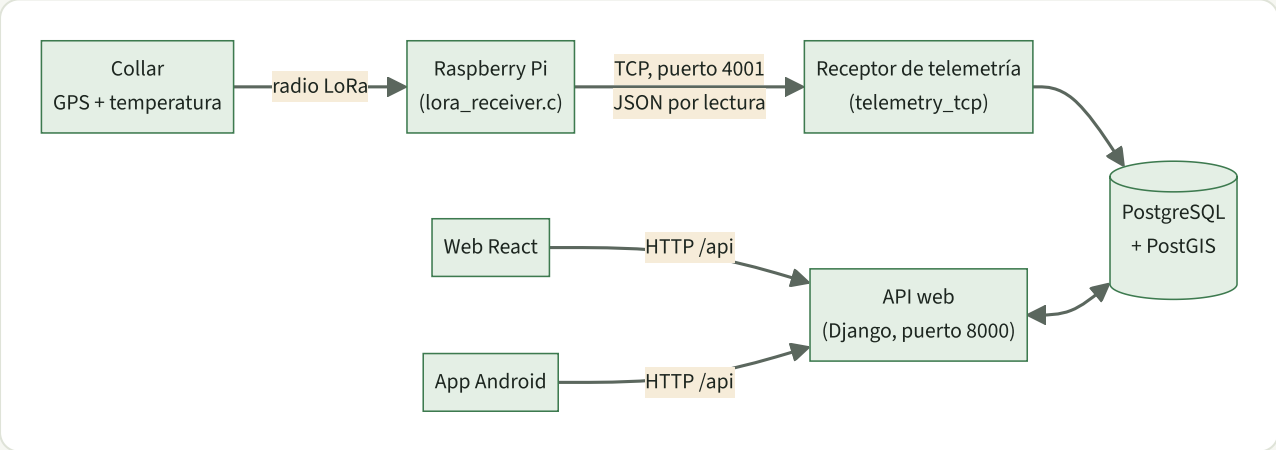
Cómo funciona el backend de MooPing

Recorrido por el servidor: cómo llegan los datos de los collares, cómo viajan por la red, cómo se guardan, cuándo se generan las alertas y cómo los muestra la app. Va lo esencial, sin entrar en detalles finos.

1 · PANORAMA

La foto completa

Cada vaca lleva un collar que manda su posición y su temperatura por radio LoRa. Una Raspberry Pi en el campo recibe esos mensajes y los reenvía al servidor en el momento, sin guardar nada. El servidor los guarda en PostgreSQL y los ofrece a la web y a la app de Android.



El backend tiene **dos puertas de entrada** que comparten el mismo código y la misma base:

- **Receptor de telemetría (TCP 4001).** Escucha a la Raspberry Pi. Solo recibe lecturas, nunca responde datos.
- **API web (HTTP 8000).** La usan la web y la app para iniciar sesión, ver el mapa, las alertas, los collares, los potreros y demás.

2 · COMUNICACIÓN

Las capas de red

Para llegar del collar a la pantalla, el dato pasa por dos redes distintas. Primero va por radio LoRa, del collar a la Raspberry Pi. Después va por internet o por la red local, con TCP/IP, de la Pi al servidor. La Pi hace de **puente**: recibe el mensaje por radio, lo abre y lo vuelve a mandar por la red.

¿Qué es LoRa?

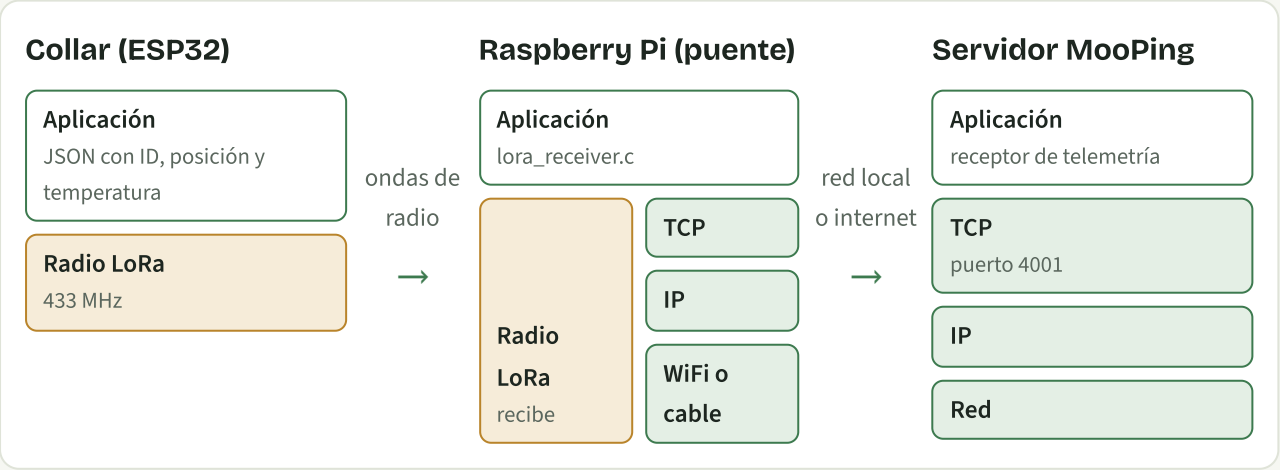
LoRa (de *Long Range*, "largo alcance") es una forma de transmitir datos por radio pensada para mandar mensajes chicos muy lejos gastando muy poca batería. Puede cubrir varios kilómetros en campo abierto, pero transmite despacio, así que solo sirve para datos cortos como una posición y una temperatura. En MooPing, el collar y la Pi usan un chip SX1278 en la frecuencia de **433 MHz**. Es LoRa "puro", es decir, sin la red LoRaWAN de terceros: el collar le habla directo a la Pi.

¿Qué es TCP?

TCP (*Transmission Control Protocol*) es el protocolo que usa casi todo internet para que los datos lleguen **completos y en orden**. Antes de mandar, abre una conexión entre los dos equipos. Si se pierde un pedazo en el camino, lo vuelve a enviar. Cada programa que escucha tiene un número de **puerto**: el receptor de telemetría escucha en el **4001** y la API web en el 8000 (443 con HTTPS en producción). TCP viaja siempre sobre **IP**, que se encarga de la dirección de cada equipo, por eso se habla de "TCP/IP".

El camino, capa por capa

Cada equipo arma el mensaje en capas: la de arriba es el dato que nos importa y las de abajo se encargan de transportarlo. El que lo recibe las va sacando en orden inverso.



El mensaje baja por las capas en el equipo que lo manda y sube por las capas en el que lo recibe. La Pi lo sube desde la radio hasta la aplicación y lo vuelve a bajar por TCP/IP.

Comparado con el modelo OSI

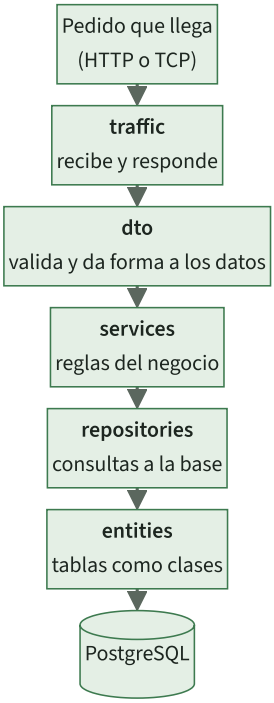
El modelo OSI divide cualquier comunicación en 7 capas. En la práctica, varias se juntan, por eso hay casilleros vacíos.

CAPA OSI	COLLAR → PI	PI → SERVIDOR	WEB / APP → SERVIDOR
7 · Aplicación el dato en sí	Mensaje JSON del collar	El mismo JSON, más la clave	API web: pedidos HTTP con JSON
6 · Presentación formato y cifrado	Texto JSON	Texto JSON (UTF-8)	HTTPS: cifrado TLS (en producción)
5 · Sesión la "charla"	—	Una conexión por lectura	Cookie de sesión del usuario
4 · Transporte que llegue entero	— (LoRa no tiene)	**TCP**, puerto 4001	**TCP**, puerto 443 u 8000
3 · Red direcciones	— (es directo, sin rutas)	**IP**	**IP**
2 · Enlace entre vecinos	Paquete LoRa con control de errores (CRC)	WiFi o Ethernet	WiFi o datos móviles
1 · Física el medio	**Radio LoRa**, 433 MHz	Ondas WiFi o cable	Ondas WiFi o antena celular

> LoRa no tiene capas de red ni de transporte. Si un paquete de radio se pierde, nadie lo reenvía: la Pi espera la próxima lectura del collar. Desde la Pi en adelante, TCP sí garantiza que el mensaje llegue completo al servidor.

Las capas

El código está separado en capas. Cada una tiene una sola tarea y solo habla con la de abajo, así un cambio en la base no obliga a tocar las pantallas y al revés.



traffic

`api/traffic/`

La puerta. Define las URLs, revisa quién es el usuario y qué puede ver, y devuelve la respuesta. También tiene el receptor TCP.

dto

`api/dto/request · response`

Los "formularios" de entrada y salida. Limpian lo que llega (por ejemplo, descartan una temperatura imposible) y arman el JSON que sale.

services

`api/services/`

Las decisiones: si una vaca salió del potrero, si tiene fiebre, quién puede editar qué. Acá vive la lógica de MooPing.

repositories

`api/repositories/`

Las consultas a la base, incluidas las geográficas con PostGIS (distancias, puntos dentro de un potrero).

entities

`api/entities/`

Cada tabla de la base representada como una clase de Python. La estructura real de las tablas la define `db/schema.sql`.

MVC y DTO

Además de las capas, el backend sigue dos patrones conocidos que ayudan a que el código sea ordenado y fácil de cambiar.

MVC: Modelo, Vista, Controlador

MVC es una forma clásica de separar una aplicación en tres partes: los **datos** (Modelo), lo que **ve el usuario** (Vista) y quien **coordina** entre los dos (Controlador). En MooPing queda así:

Modelo

`entities + repositories + services`

Los datos y sus reglas: animales, collares, alertas, y la lógica que decide qué hacer con ellos. Vive en la base de datos y en las capas de abajo del backend.

Vista

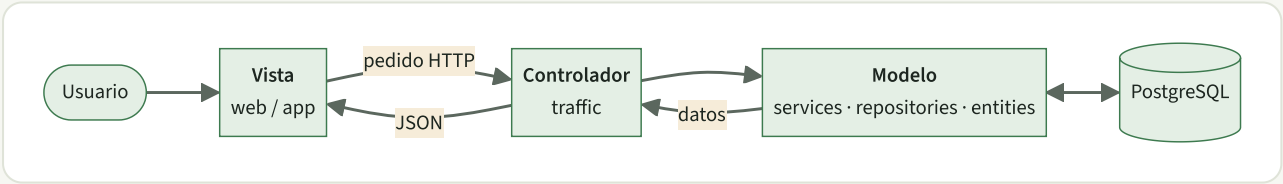
`web React + app Android`

Las pantallas: el mapa, el Centro de Alertas, Equipos. Le piden datos al servidor y los dibujan. El servidor solo les entrega JSON.

Controlador

`traffic/*_views.py`

Recibe cada pedido, revisa permisos, llama al servicio que corresponde y devuelve la respuesta. No tiene lógica de negocio ni consultas a la base.

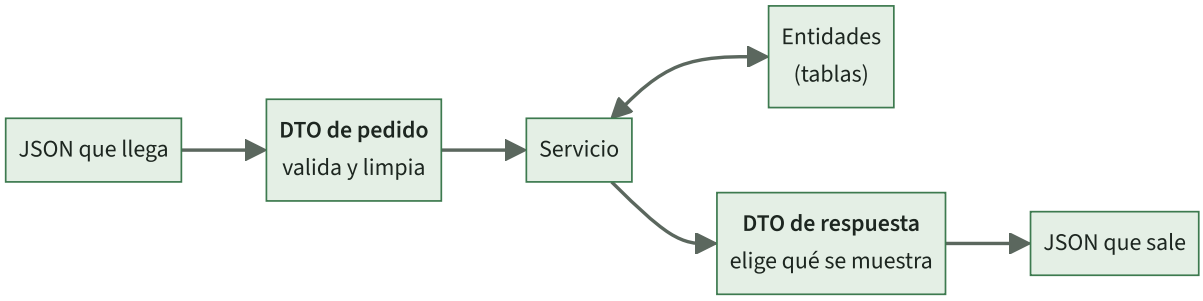


Django llama a este patrón "MTV" y a sus controladores les dice "views". Por eso los archivos se llaman `*_views.py` aunque cumplen el papel del Controlador.

DTO: objetos para transportar datos

Un **DTO** (*Data Transfer Object*) es un objeto simple que solo sirve para **llevar datos de una capa a otra**, sin lógica adentro. En MooPing todo lo que entra y todo lo que sale pasa por un DTO. Hay dos tipos:

- **DTO de pedido** (`dto/request/`). Recibe lo que manda la app o la Pi, lo valida y lo deja limpio. Por ejemplo, la lectura de un collar acepta `LAT` , `lat` o `latitude` y siempre la entrega como un número.
- **DTO de respuesta** (`dto/response/`). Define exactamente qué campos salen en el JSON. Así nunca se manda una tabla de la base tal cual, con datos internos como el hash de la contraseña.

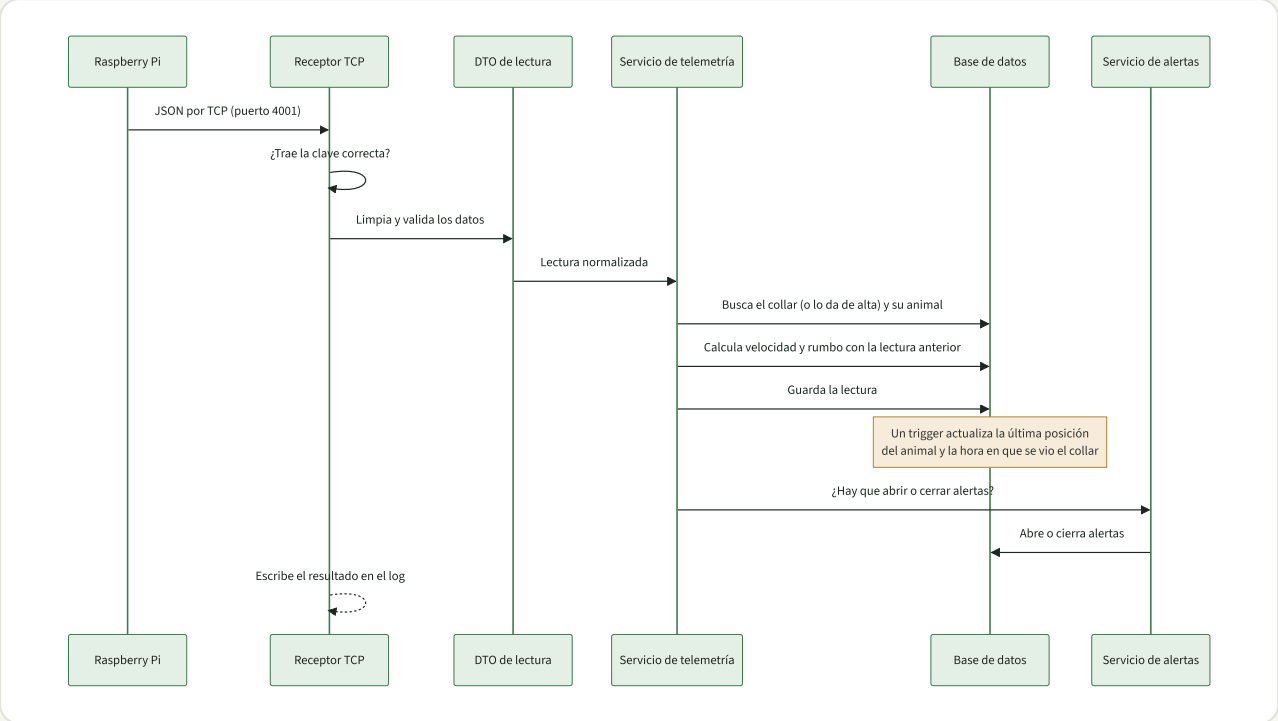


> La ventaja es que si cambia una tabla de la base, alcanza con ajustar el DTO. La app sigue recibiendo el mismo JSON y no se rompe.

5 · TELEMETRÍA

El viaje de una lectura

Esto es lo que pasa desde que la Raspberry Pi manda un mensaje como `{"ID":"COLLAR-3","LAT":-34.6,"LONG":-58.4,"TEMP":38.7}` hasta que queda guardado.



Qué se revisa en cada lectura

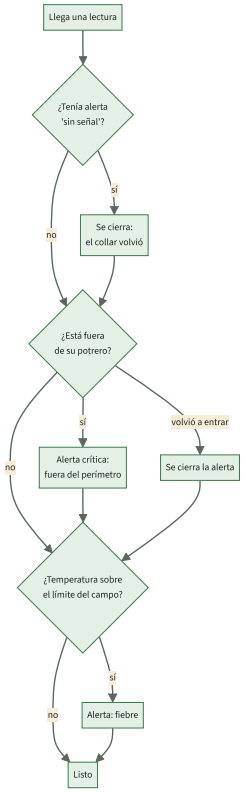
- **Clave.** Si el servidor tiene configurada una clave, la lectura tiene que traerla. Si no la trae, se descarta. Así nadie de afuera puede inventar posiciones.
- **Collar.** Tiene que venir el ID. Si el collar es nuevo, se registra solo y queda esperando a que alguien lo asigne a un animal.
- **GPS sin señal.** Si la posición llega en 0,0, se guarda la lectura sin ubicación.
- **Sensores raros.** Una temperatura fuera de 0 a 60 °C se ignora. El resto de la lectura se guarda igual.
- **Hora.** Si el reloj de la Pi está muy desfasado, se usa la hora de llegada al servidor.

> La misma lógica también está disponible por HTTP, en `POST /api/telemetry/`, para cuando la Pi tenga que mandar por internet en vez de por la red local.

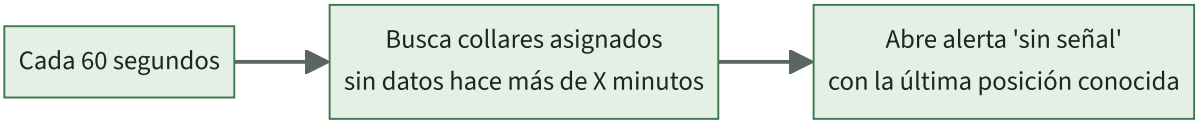
6 · ALERTAS

Cuándo se genera una alerta

Cada lectura de un collar asignado a un animal pasa por estas preguntas. Las alertas se abren solas y también se cierran solas cuando el problema desaparece.



La alerta **"sin señal"** funciona al revés, porque se dispara justamente cuando no llega nada. El receptor TCP tiene un proceso aparte que cada minuto busca collares asignados que hace rato no reportan.

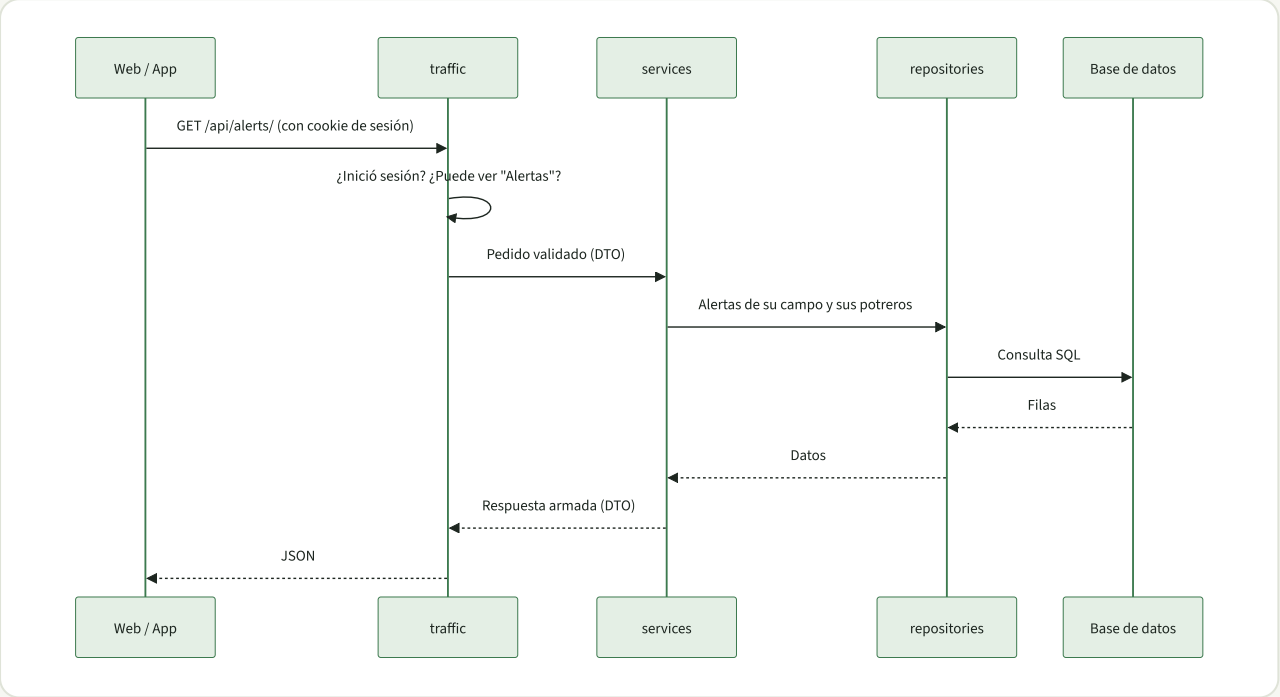


El tiempo de espera (X) y el límite de fiebre se configuran por establecimiento.

7 · API WEB

Cuando la app pide datos

La web y la app de Android le piden datos a la API cada pocos segundos. Por ejemplo, el Centro de Alertas consulta cada 15 segundos. Cada pedido pasa por las mismas capas:



Quién ve qué

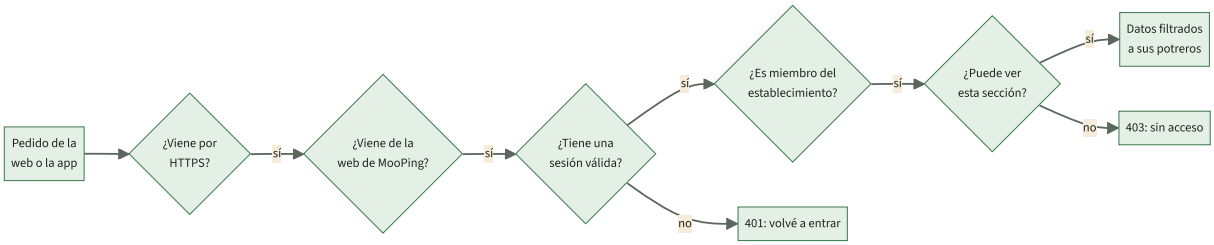
- Cada usuario pertenece a un **establecimiento** y solo ve los datos de ese campo.
- El **dueño** ve todo y administra. Los **miembros** pueden quedar limitados a ciertas secciones, potreros o animales.
- La sesión es una cookie con un token que se guarda en la base (tabla `api_tokens`).

GRUPO DE RUTAS	PARA QUÉ SIRVE
`/auth, /me`	Iniciar y cerrar sesión, cambiar la contraseña
`/organization`	Datos del establecimiento y sus usuarios
`/dashboard, /monitoring…`	Resúmenes para las pantallas Inicio, Monitoreo y Equipos
`/alerts`	Listar, atender y comentar alertas
`/animals`	Animales, posiciones en el mapa, recorridos y fotos
`/yards`	Potreros (los perímetros dibujados en el mapa)
`/devices, /gateways`	Collares, estaciones y repetidoras
`/support, /products`	Soporte técnico y tienda
`/telemetry`	Entrada de lecturas por HTTP (alternativa al TCP)

8 · SEGURIDAD

Cómo se protege

La seguridad funciona como una serie de filtros. Cada pedido tiene que pasarlos todos antes de llegar a los datos, y si falla uno se corta ahí.



Resumen de las protecciones

Contraseñas

Nunca se guardan tal cual. Se guarda un **hash**, una huella que no se puede revertir. Al entrar se compara la huella. Hay un **límite de intentos** de login por equipo y por cuenta, para frenar a quien quiera adivinar claves.

Sesión

Al entrar, el servidor genera un código al azar y lo guarda en una **cookie protegida** que el código de la página no puede leer. Dura 30 días. En la base solo queda su huella, y al cambiar la contraseña se cierran las otras sesiones.

Permisos

Cada usuario solo ve su **establecimiento**. El dueño ve todo. A los demás miembros se les puede limitar qué secciones abren y qué potreros o animales ven, y ese filtro se aplica en las consultas a la base.

Ataques desde otros sitios

El servidor solo acepta pedidos que modifican datos si vienen de la web de MooPing (CORS y un encabezado especial). Así otra página no puede usar la sesión del usuario a escondidas.

Telemetría

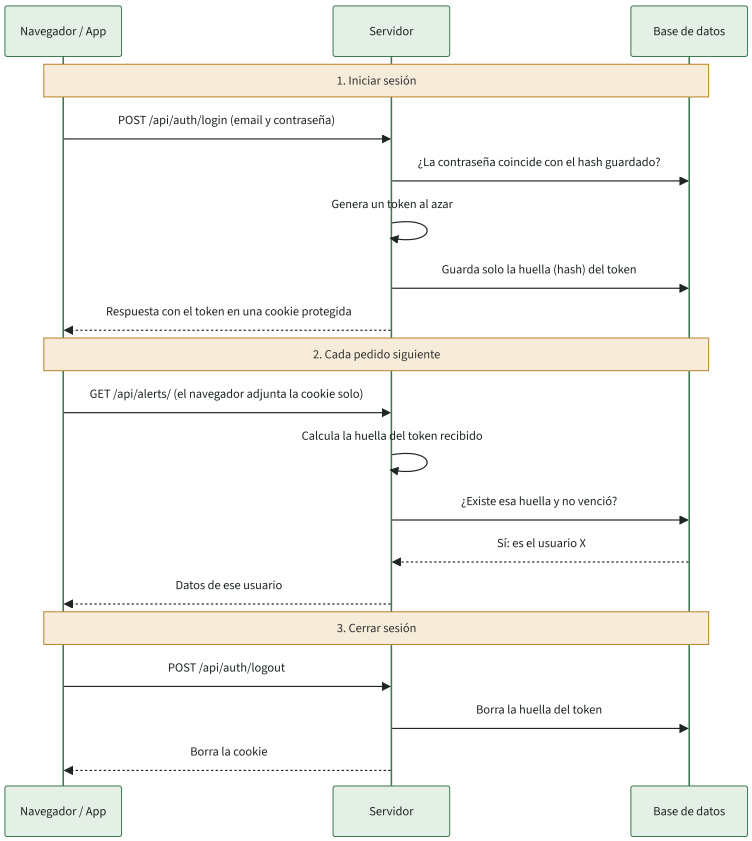
La Pi tiene que mandar una **clave compartida** en cada lectura. Si no coincide, se descarta. El receptor también limita cuántas conexiones acepta y cuánto tiempo las deja abiertas, para que no lo puedan saturar.

Producción

Caddy pone **HTTPS**, así todo viaja cifrado, y agrega encabezados que le dicen al navegador qué puede cargar la página. La base de datos no queda expuesta a la red, solo la usa el backend.

Cómo viaja el token de sesión

El **token** es un código largo y al azar que funciona como una pulsera de entrada: se entrega una vez al iniciar sesión y después se muestra en cada pedido. Este es su recorrido:

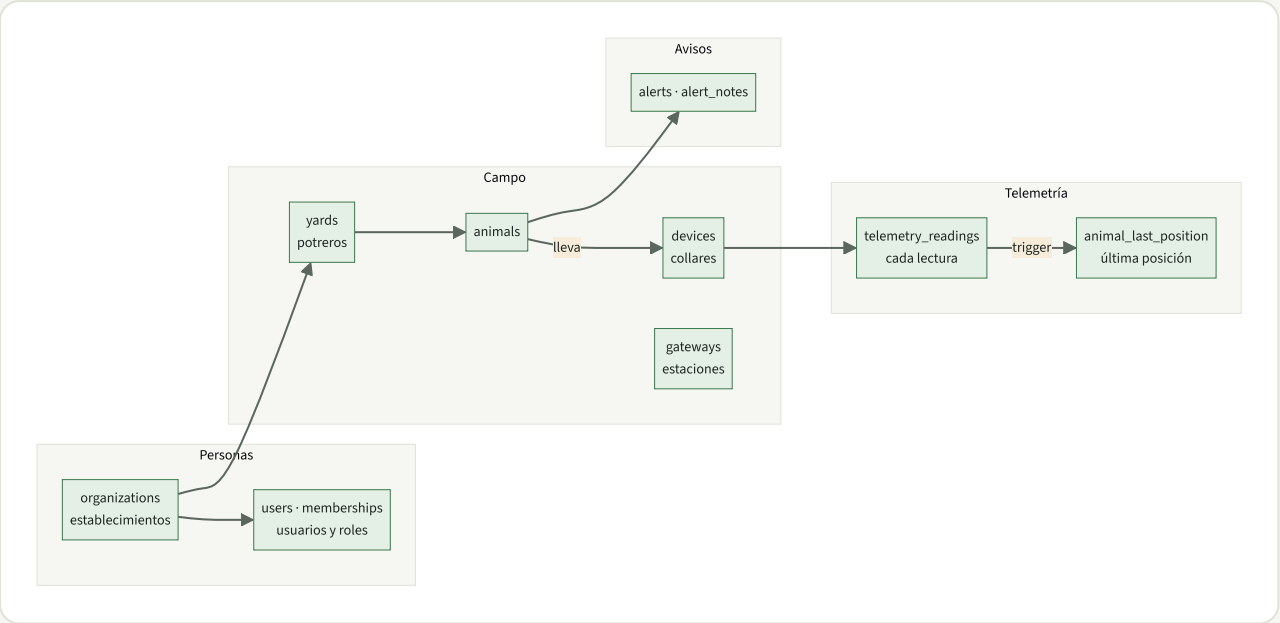


- **Viaja en una cookie** llamada `mooping_session`. El navegador la manda solo en cada pedido a `/api/`, así que el código de la página nunca tiene que tocarla.
- **Nadie la puede leer desde la página** (cookie *HttpOnly*). Si alguien lograra meter código malicioso en la web, igual no podría robar la sesión.
- **En producción solo viaja por HTTPS** (cookie *Secure*), así no se puede espiar en el camino.
- **La base guarda la huella, no el token.** Si alguien copiara la base, no podría usar esas huellas para entrar.
- **Vence a los 30 días.** También se invalida al cerrar sesión, y al cambiar la contraseña se cierran las demás sesiones abiertas.
- **La app de Android funciona igual.** Por dentro es un navegador (WebView) que guarda la cookie de la misma forma.

9 · DATOS

La base de datos, por grupos

Es PostgreSQL con la extensión PostGIS, que sirve para guardar puntos y polígonos y calcular distancias. Las tablas se agrupan así:

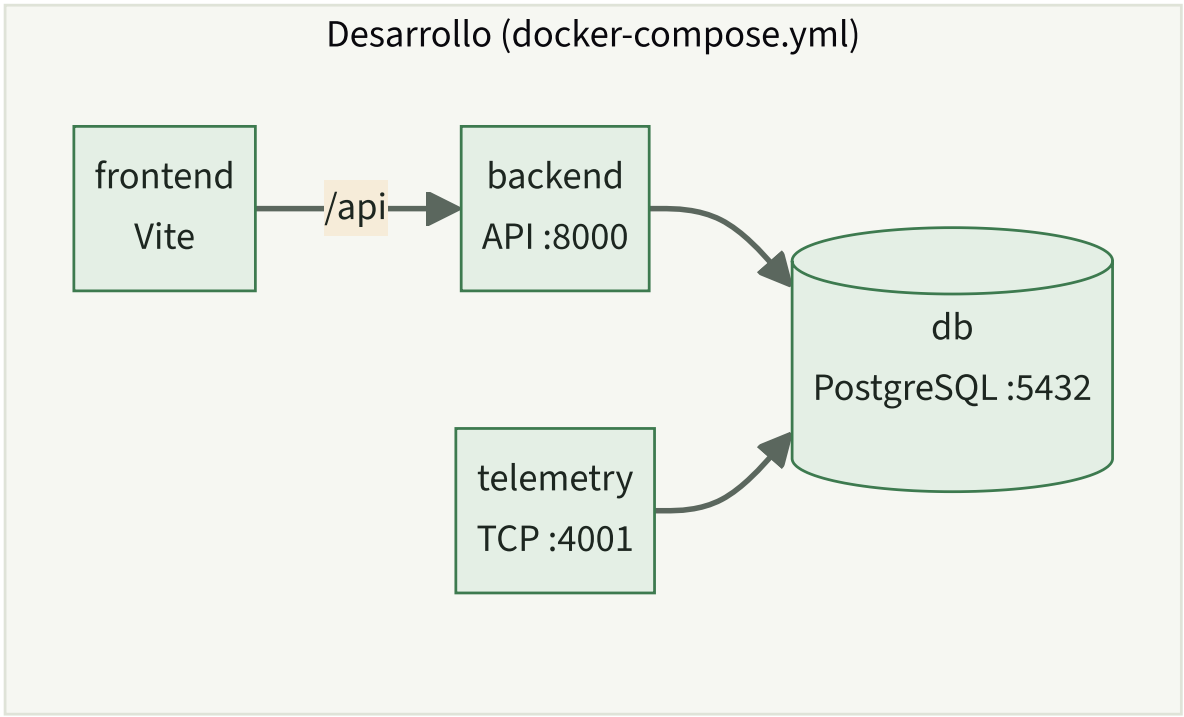


Aparte hay tablas de apoyo: permisos por potrero o animal, sesiones, ajustes de notificaciones, comandos y mantenimiento de collares, y pedidos de soporte.

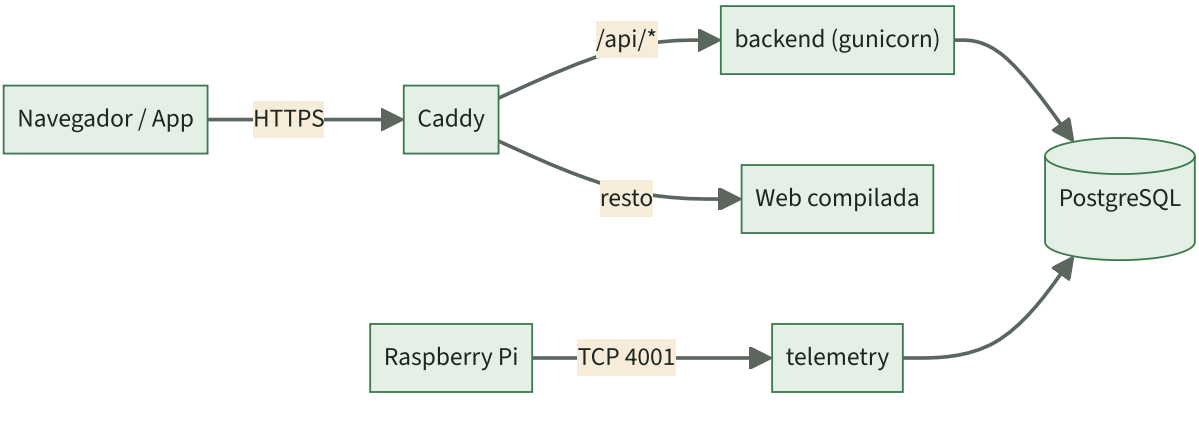
> `telemetry_readings` guarda la historia completa y crece con cada lectura. `animal_last_position` tiene una sola fila por animal, así el mapa carga rápido sin recorrer toda la historia.

Cómo se levanta

Todo corre en contenedores Docker. Con `docker compose up` se levantan estas piezas:



En producción se suma **Caddy** adelante. Pone HTTPS, sirve la web ya compilada y pasa todo lo que empieza con `/api` al backend. El puerto TCP de telemetría queda aparte, para la Raspberry Pi.



Basado en el código de `mooping/backend` y `mooping/db/schema.sql`. Noviembre de 2026.